

Test Project

Module A Mini Speed Test Projects

Web Technologies

Submitted by:
SamsungSkills

Contents

Contents	2
Introduction	3
Description of project and tasks	3
Part A: Design Implementation	3
A1 Traffic Light (Level 1)	3
A2 Loading Animation (Level 1)	3
A3 Moon Orbit (Level 2)	3
A4 Text Slide Animation (Level 2)	4
A5 3D Chart (Level 3)	4
Part B: Front-End	4
B1 POS Quantity & Price Calculator (Level 1)	4
B2 Client-side Validation (Level 1)	4
B3 Sortable (Level 2)	5
B4 Mini Calendar (Level 2)	5
B5 Building Floor Plan Pathfinder (Level 3)	5
Part C: Back-End	6
C1 Coupon Code Validation (Level 1)	6
C2 Stock Deduction (Level 1)	6
C3 Leaderboard (Level 2)	6
C4 Color Channel Filter (Level 2)	6
C5 Library SQL Queries (Level 3)	7
Instructions to the Competitor	7
Other	7
<i>Marking Summary</i>	<i>7</i>

Introduction

In this mini speed test project, you are given multiple mini tasks. Please only implement the required tasks.

For each mini task, there are three levels.

- Level 1: 0.50 points, expected to take less than 5 minutes of work.
- Level 2: 0.75 points, expected to take less than 10 minutes of work.
- Level 3: 1.00 points, expected to take around 20 minutes of work.

Please put your files in /XX_module_a/ with the task ID as the folder name, e.g. /XX_module_a/A1/. The XX is your workstation number.

Description of project and tasks

Please create an index page to link to each mini speed Test Project.

The index page should contain thumbnail and title of each mini speed test project.

Please write at least one git commit for each mini speed Test Project. The git commit messages should be readable and easy to understand.

Part A: Design Implementation

Use only HTML and CSS. Do not use JavaScript.

A1 Traffic Light (Level 1)

- Build a vertical traffic light with three lamps (green, yellow, red).
- Display a countdown number next to the currently active lamp: green 5s → yellow 3s → red 5s, looping continuously.
- Only the active lamp is lit and shows its countdown number; inactive lamps remain dim and show no number.

A2 Loading Animation (Level 1)

- Create a circular spinner that rotates 360 degrees in an infinite loop.
- Only part of the spinner's border is colored so the rotation direction is visually clear.
- The rotation speed is 1 second per full revolution.

A3 Moon Orbit (Level 2)

- Place the sun, earth, and moon using the provided sprite image (sprite.png).
- The earth orbits the sun (8 seconds per loop), and the moon orbits the earth (2 seconds per loop).
- Both orbits run simultaneously in an infinite loop.

Provided file: sprite.png

A4 Text Slide Animation (Level 2)

- Reproduce the same animation shown in the provided video (animation.mp4).
- Display the following 3 lines of text at the center of the screen: "Hi, nice" / "to see" / "you here"
- Each line slides down from above while fading in at the same time.
- The lines fade in with a staggered delay, in the order: third line → second line → first line.
- After being displayed for a moment, the lines fade out in the same order.
- The last line ("you here") is displayed in a different (accent) color.

Provided file: animation.mp4

A5 3D Chart (Level 3)

- Recreate the 3D bar chart shown in the provided reference image (chart.png), with the same values (60, 25, 35, 80).
- Each bar should have a 3D appearance (top face and side face included).
- On page load, each bar should animate by filling up from the bottom to its corresponding value.

Provided file: chart.png

Part B: Front-End

You may use JavaScript, HTML, and CSS. Backend languages are not allowed.

B1 POS Quantity & Price Calculator (Level 1)

- In the provided screen, implement the +/- buttons so that the total (unit price × quantity) updates automatically.
- The quantity must not go below 0.

Provided file: starter HTML

B2 Client-side Validation (Level 1)

- Implement input validation for the provided contact form (name, email, message).
- The name and message fields must not be empty.
- The email must be in a valid format (must contain @ and . , etc.).
- If any condition is not met, the form must not be submitted, and appropriate feedback must be shown to the user.
- If all conditions are met, the user must be informed that the submission was successful.

Provided file: starter HTML

B3 Sortable (Level 2)

- Implement the following features for the provided To Do / Done lists.
- The user can reorder items within each list.
- The user can move items between the two lists.

Provided file: starter HTML

B4 Mini Calendar (Level 2)

- The provided screen shows an example calendar for June 2026, including some dates from the previous and next months.
- Implement the "previous month" / "next month" buttons so that clicking them switches the calendar to the corresponding month.
- The switched calendar must also correctly place dates according to the day of the week, including dates from the previous and next months.
- The current date according to the system must always be highlighted (this highlight should not appear when viewing a different month).

Provided file: starter HTML

B5 Building Floor Plan Pathfinder (Level 3)

- In the provided floor plan (rooms A–H, connected by corridors with weights), the user selects a start and end room by clicking on rooms.
- The first click sets the start room, and the second click sets the end room. The shortest path (minimum total weight) between the two rooms is then highlighted on the floor plan.
- The third click behaves the same as the first click: the clicked room becomes the new start room, and any previously shown end room and highlighted path disappear.
- The result is displayed in the panel at the top, in the same format as the example in the starter HTML (A → F: A-D-E-F, total distance 7).

Test cases for verification

Start	End	Shortest path	Total distance
A	F	A-D-E-F	7
A	G	A-D-E-F-G	8
D	C	D-E-F-C	6
A	H	A-D-E-F-G-H	12

Provided file: starter HTML

Part C: Back-End

Use a backend language and a database.

The API endpoint for each task follows the format {host}/XX_module_a/{task ID} (e.g., the endpoint for C1 is {host}/XX_module_a/C1, where “XX” is your workstation number).

C1 Coupon Code Validation (Level 1)

- Accept a coupon code string via the GET parameter code (e.g., ?code=SAVE2024).
- A valid coupon code format consists of 4 uppercase letters followed by 4 digits (8 characters total), e.g., SAVE2024.
- If the input matches this format, return {"valid": true} as JSON; otherwise, return {"valid": false}.

C2 Stock Deduction (Level 1)

- Use the provided C2_products table (product IDs 1–3, with stock 10 / 50 / 0 respectively).
- Accept `product\_id` and `quantity` via POST.
- If the stock is greater than or equal to `quantity`, deduct the stock and return `{"success": true, "remaining\_stock": N}`.
- If the stock is less than `quantity`, do not deduct and return `{"success": false, "remaining\_stock": N}` (where `N` is the stock before deduction).
- Example: a request with `product\_id=1, quantity=3` returns `{"success": true, "remaining\_stock": 7}`.

Provided file: SQL (schema.sql)

C3 Leaderboard (Level 2)

- Use the provided `C3\_scores` table (includes 5 sample players; Bob and Charlie have the same score).
- Accept `player\_name` and `score` via POST to register a new score.
- On a GET request, return the full leaderboard sorted by score in descending order. In case of a tie, the player registered earlier ranks higher.
- The leaderboard must be returned as a JSON array containing `rank`, `player\_name`, and `score`.

Provided file: SQL (schema.sql)

C4 Color Channel Filter (Level 2)

- Use the provided image (sample.jpg).
- Accept a GET parameter `channel` with one of the values `red`, `green`, or `blue` (e.g., `?channel=red`).
- Return an image where the selected channel keeps its original values, and the other two channels are set to 0 (e.g., if `channel=red`, the R values remain unchanged while G and B become 0).
- The response must be returned directly as an image (PNG or JPEG).

Provided file: sample.jpg

C5 Library SQL Queries (Level 3)

- Use the provided library database (`C5\_books`, `C5\_members`, `C5\_loans`).
- Accept a GET parameter `query` with a value of 1, 2, or 3 (e.g., `?query=1`), and return the result for the corresponding question as a JSON array. (Each question is worth 0.5 points, for a total of 1.5 points.)
 1. Return the number of loans (`loan\_count`) per member, sorted in descending order. (`name`, `loan\_count`)
 2. Return the title and borrower's name for books that are currently on loan (not yet returned). (`title`, `member\_name`)
 3. Return the number of books (`book\_count`) per category. (`category`, `book\_count`)

Provided file: SQL (schema.sql)

Instructions to the Competitor

Each mini Test Project folder should be separated and self-contained.

Please include only exactly the required mini test project.

And do not include any not related folders or not related mini Test Projects.

Other

This project will be assessed by using Google Chrome web browser.

Marking Summary

	Sub-Criteria	Marks
1	Mini Test Project General	1.5
2	Design Implementation	3.5
3	Front-end Development	3.5
4	Back-end Development	3.5